法币支付：金额加密、第三方调用与熔断幂等

gomall /paydown 业务侧 deck

gomall · 业务侧 deck

今日大纲

支付链路全景与四个硬约束

PayDown 主体：四件事必须原子

第三方支付与熔断

幂等：客户端必须带原 Idempotency-Key 重试

支付失败 Saga + 实测数字 + 业务承诺

支付链路全景与四个硬约束

/paydown 在 gomall 的位置

- 用户视角：只是一次”点确认”。
- 系统视角：一次 `POST /paydown` 必须**同时**做对四件事：
  - 解密买家密文金额、扣款、加密回写。
  - 解密卖家密文金额、入账、加密回写。
  - DB 扣库存 + 翻订单状态到 `PendingShipping`。
  - 写 `order.paid` outbox 事件。
- 任一步失败：全部回滚 + 客户端可重试 + **绝不扣两次**。

service/payment.go:35–200；routes/router.go:114–121

业务侧的四个硬约束

约束	设计回应	代码位置
金额不能明文落盘	AES-128-CBC 加密	user.go:59–79
密码错不能扣款	长度 + 解密失败即 abort	payment.go:42–45
第三方支付会挂	CircuitBreaker 5/10s/3	circuitbreaker.go
客户端会重试	Idempotency-Key + Lua	idempotency.go

四条由**合规 + 客诉 + 监控**三方倒推。后面四节按此顺序展开。*routes/router.go:114–121 paydown* 完整中间件栈

明文存金额会出什么事

- **合规**：等保 2.0 / PCI-DSS 要求”敏感字段不得明文落盘”，金额属于此类。
- **防内鬼**：DBA / 离职工程师拿到一次 dump → 全站余额裸了。
- **防误日志**：明文金额一旦被 `log.Debug(user)` 序列化 → 上传到 ELK。
- **审计**：金额被 SQL 直改 → 密文校验失败可立刻发现。

> 加密金额不是”加密功能”，是**把金额从数据库视野里抹掉**。运维看到的永远是乱码。

gomall 的金额加密链路

- 双密钥：服务端 `MoneySecret` + 用户支付密码 `key`。
- 任一方泄漏**都不足以**独立解密 □ 密钥分离。
- 改密码 = 解密旧密文 + 用新 key 重新加密。

repository/db/model/user.go:59–79

加解密的代码长这样

Listing 1: *repository/db/model/user.go:59–79*

```go
// EncryptMoney 加密金额
func (u *User) EncryptMoney(key string) (money string, err error) {
   aesObj, err := secret.NewAesEncrypt(
      conf.Config.EncryptSecret.MoneySecret, // 服务端持有
      key, // 用户支付密码
      "", secret.AesEncrypt128, secret.AesModeTypeCBC)
   if err != nil { return }
   money = aesObj.SecretEncrypt(u.Money)
   return
}

// DecryptMoney 返回值单位为分
func (u *User) DecryptMoney(key string) (money int64, err error) {
   aesObj, _ := secret.NewAesEncrypt(
      conf.Config.EncryptSecret.MoneySecret, key, "",
      secret.AesEncrypt128, secret.AesModeTypeCBC)
   money = cast.ToInt64(aesObj.SecretDecrypt(u.Money))
   return
}
```

逐行讲解：加解密做了什么

- **61 行**：服务端密钥 + 用户密码当两个独立输入，任一方旋转都让原密文失效。
- **64 行 CBC 而非 ECB**：同一笔金额每次密文不同，防” 对照表” 猜金额。
- **65 行 `Money string`**：密文是 base64 长度可变，直接存 string。
- **77 行 `cast.ToInt64`**：解密回 int64，**单位是分**——全链路避免浮点。
- **错误传播**：密码错 → 乱码 → `ToInt64` 返 0 → 金额校验失败 → TX 回滚。

repository/db/model/user.go:59–79

支付密码 vs 登录密码：为什么分两套

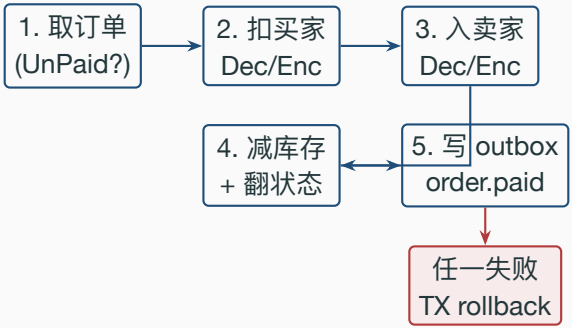
	登录密码	支付密码
作用	拿 JWT	解密金额
长度	≥ 6	固定 6 位
泄漏	看订单地址	直接扣款
隔离	bcrypt 单向	AES 双向
存储	哈希	不存 (仅入参)

- 登录密码哈希存：服务端只校验、不还原。
- 支付密码**不存**：用户每次手输 □ 整库 dump 也解不开。
- 入口校验 `len(req.Key) == 6`，太短直接拒。

service/payment.go:42–45；consts/user.go:35

PayDown 主体：四件事必须原子

PayDown 在一次事务里做的五件事



五步在同一 `tx`，**要么全成功要么全不动。**

outbox 也在 TX 内写——事务消息防丢核心（deck 04）。*service/payment.go:52–184*

PayDown 的关键 18 行

Listing 2: *service/payment.go:55–96* 节选

```go
order, err := dao.NewOrderDaoByDB(tx).GetOrderById(req.OrderId, uId)
if err != nil { return err }
if order.Type != consts.UnPaid {
   return errors.New("订单状态非未支付，无法重复支付")
}
totalMoney := order.Money * int64(num)

user, err := userDao.GetUserById(uId)
if err != nil { return err }

userMoney, err := user.DecryptMoney(req.Key) // 用支付密码解密
if err != nil { return err }
if userMoney-totalMoney < 0 {
   return errors.New("金额不足") // TX 内，自动回滚
}

user.Money = strconv.FormatInt(userMoney-totalMoney, 10)
user.Money, err = user.EncryptMoney(req.Key) // 同密码重新加密
if err != nil { return err }
err = userDao.UpdateUserById(uId, user) // 落 DB
```

逐行讲解：扣款这一步的三个要点

- **57 行 UnPaid 校验在 TX 内**：不能 TX 外读再进事务——否则典型 TOCTOU，两并发支付都看到 UnPaid → 双扣。
- **65 行 `DecryptMoney(req.Key)`**：密码错 → 解出乱码 → `userMoney` 几乎一定不够 → 走”金额不足”分支。**密码错和余额不足表象一致**，防穷举。
- **71–72 行重新加密**：`Money` 是 string 不能直接减；必须 decrypt → int64 减 → encrypt。整段在 TX 里，失败回滚干净。
- **金额不足也 `return err`**：让 TX 走 rollback，`order.Type` 不会推进，客户端可原 OrderId 重试。

service/payment.go:55–96

outbox 写在事务尾

Listing 3: *service/payment.go:174–184*

```
// outbox 事件：order.paid，投递交给 publisher 异步
return dao.NewOutboxDaoByDB(tx).Insert(
   "order", "OrderPaid", "order.paid", order.ID,
   events.OrderPaid{
      OrderID: order.ID, OrderNum: order.OrderNum,
      UserID: uId, ProductID: productID, Num: num,
   },
)
```

`outbox.Insert` 用的是**同一个 tx** ——事务消息：扣款与”已发事件”同生共死，**绝不会”钱扣了但下游没收到”**。Redis 预占释放走 TX 外，失败仅记日志，对账兜底。

service/payment.go:193–197 cache.CommitReservation

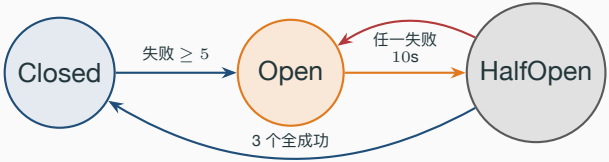
第三方支付与熔断

当前 gomall 的” 第三方支付” 是什么

- gomall 当前未接真实支付宝/微信，” 扣买家、入卖家” 在本地 DB 完成。
- 但 `/paydown` 已挂 `CircuitBreaker` ——**给未来留位置**：
  - 接入真实网关后链路变成”DB tx + 同步 RPC 第三方”。
  - 第三方是慢依赖：p99 数百 ms + 会” 集体抽风”。
  - 不熔断 → goroutine 堆积 → 进程 OOM → 全站崩。
- 熔断作用：**宁可 5% 用户立即看到” 稍后重试”，也不让 100% 用户白屏 30s。**

routes/router.go:114–121；*service/payment.go* 当前实现

CircuitBreaker 的三态



状态	含义	用户体感
Closed	正常放行	正常下单
Open	全拒、立即返回	立刻 70002
HalfOpen	放 3 个探测	部分通过

routes/router.go:115–119

middleware/circuitbreaker.go:17–21；

allow() ——决定能否放行

Listing 4: *middleware/circuitbreaker.go:76–106*

```go
func (cb *circuitBreaker) allow() error {
   switch circuitState(cb.state.Load()) {
   case stateClosed:
      return nil // 热路径零开销
   case stateOpen:
      if time.Now().UnixNano()-cb.openedAt.Load() < int64(cb.opt.OpenTimeout) {
         return errCircuitOpen // 未到自愈窗口
      }
      cb.mu.Lock(); defer cb.mu.Unlock()
      if circuitState(cb.state.Load()) == stateOpen { // double-check
         cb.state.Store(int32(stateHalfOpen)) // Open -> HalfOpen
         cb.halfOpenReq.Store(0)
      }
      if cb.halfOpenReq.Add(1) > cb.opt.HalfOpenMaxReq {
         return errCircuitOpen // 探测配额满
      }
      return nil
   }
   return nil
}
```

逐行讲解：allow() 的关键决策

- **Closed 分支零开销**：只一次原子 Load → return nil，不抢锁不计数。这是熔断器能放 paydown 入口的前提。
- **Open 早返回**：`time.Now() - openedAt` 无锁原子读，99% 的 Open 期请求从这里返回。
- **加锁切 HalfOpen**：只有自愈窗口的**第一个**请求加锁切状态——double-check 防两 goroutine 同时切。
- **HalfOpen 探测配额**：`halfOpenReq.Add(1) > 3` 保证半开期最多放 3 个试水。
- **report() 配套**：3 个全成功 → 回 Closed；任一失败 → 立即重新 Open，时钟重置。

middleware/circuitbreaker.go:76–128

失败的定义 + 阈值 5 / 10s / 3

Listing 5: *middleware/circuitbreaker.go:69–70*

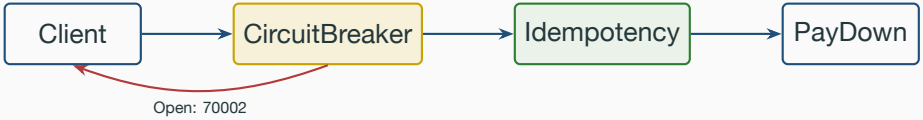
```
failed := len(c.Errors) > 0 || c.Writer.Status() >= http.StatusInternalServerError
cb.report(failed)
```

- **只看 5xx 不看 4xx**：4xx 是密码错/余额不足等业务正常路径，不熔断。200 + 业务码 70001/70002 也不算失败。
- **阈值 5**：1–2 次抖动不算挂，连续 5 次 = 真挂。
- **OpenTimeout 10s**：第三方支付重启通常 5–30s，10s 是中位数。
- **HalfOpenMaxReq 3**：试 3 个够判定，再多浪费用户体验。

routes/router.go:115–119 当前硬编码 *5/10s/3*

幂等：客户端必须带原 Idempotency-Key 重试

为什么熔断器之后还要挂幂等



- 熔断器关心**保护下游**（5xx 多了断开）。
- 幂等关心**保护用户钱包**（同一笔单只扣一次）。
- 目标正交，必须**同时**挂。返回 `200+status=70002`，HTTP 仍是 200 ▯ app 端老网关不会按 5xx 处理。

routes/router.go:114–121；pkg/e/code.go:58 ErrCircuitOpen=70002

客户端的重试时序（关键约定）

重试必须带原 Idempotency-Key ——万一原请求其实成功（CB 误判 / 网络分区），第二次会被 Idempotency 拦下，返回**原结果**，绝不二次扣款。*middleware/idempotency.go:96–110*

Idempotency 中间件的 responseRecorder

Listing 6: *middleware/idempotency.go:17–31, 96–110*

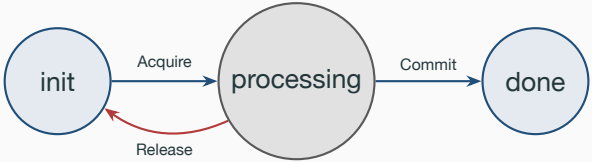
```go
// responseRecorder 拷贝写入的响应体，便于幂等命中时回放
type responseRecorder struct {
    gin.ResponseWriter
    body *bytes.Buffer
}
func (r *responseRecorder) Write(b []byte) (int, error) {
    r.body.Write(b) // 旁路缓存
    return r.ResponseWriter.Write(b) // 同时真写出去
}

// 主流程末尾:
recorder := &responseRecorder{ResponseWriter: c.Writer, body: bytes.NewBuffer(nil)}
c.Writer = recorder
c.Next()
if len(c.Errors) > 0 || recorder.Status() >= http.StatusBadRequest {
    cache.ReleaseIdempotencyLock(c.Request.Context(), key) // 失败: 回 init
    return
}
cache.CommitIdempotencyResult(c.Request.Context(), key, recorder.body.String())
```

逐行讲解：responseRecorder 解决了什么

- **嵌入 gin.ResponseWriter**：包一层而非替换，`Header/Status/Hijack` 透明转发，链路下游无感。
- **双写**：先写 `bytes.Buffer` 再调原 `Writer.Write` ——顺序很重要，反过来客户端断连缓存会写不完。
- **Hook 时机**：在 `c.Next()` 之前替换 Writer，业务函数写入时被拦截。
- **失败回滚**：4xx 或 `c.Errors` □ 回退到 init □ 同 IK 还能再试（业务失败 ≠ 永久失败）。
- **成功提交**：把完整 JSON 写进 Redis done 态 □ 后续同 IK 由 Lua 直接回放，业务代码不再执行。

middleware/idempotency.go:17–112

Lua 脚本里的四态机 + 业务码地图

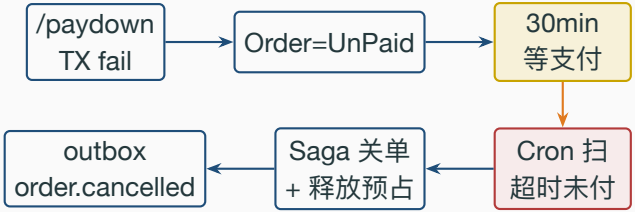


状态码	含义	客户端动作	客服话术
60001	IK 不存在/过期	重新申领	”刷新重试”
60002	幂等命中处理中	等 1s 再试	”请稍后”
70002	熔断 Open	指数退避 + 带原 IK	”支付暂忙”

repository/cache/idempotency.go:32–50；pkg/e/code.go:53–58

支付失败 Saga + 实测数字 + 业务承诺

支付失败链路——Saga 回滚



- 支付失败 = TX 回滚 = `order.Type` 仍 UnPaid，库存**从未真扣**。
- 30min 未续支付 → Cron → `CancelUnpaidOrder`。
- Saga 不回写 `product.Num`（从未扣过），仅释放 Redis `reserved`。

service/order_cancel.go:14–62；与 deck 09 / 15 衔接

CancelUnpaidOrder 的幂等设计

Listing 7: *service/order_cancel.go:30–51*

```go
err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
   ok, err := dao.NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
   if err != nil { return err }
   if !ok { return nil } // 已被别处关过, no-op
   closed = true
   return dao.NewOutboxDaoByDB(tx).Insert(
      "order", "OrderCancelled", "order.cancelled", order.ID,
      events.OrderCancelled{
         OrderID: order.ID, OrderNum: orderNum,
         UserID: order.UserID, ProductID: order.ProductID,
         Num: order.Num, Reason: "timeout",
      },
   )
})
```

CloseOrderWithCheck 用 `WHERE type=UnPaid` 条件 update，第二次调用自然 affected=0

- 函数变 no-op。这是**用 DB 行做幂等键**。Redis 释放走 TX 外，失败仅记日志。

基线对照：两个中间件接入后的吞吐

链路	RPS	p95
/ping 基线 (无中间件)	64,254	3.51 ms
/product/show (无 CB / 无 Idem)	62,226	3.01 ms
/orders/create (**Idempotency**)	50,319	2.33 ms
/skill_product/skill (RateLimit)	52,082	1.24 ms

- Idempotency 让吞吐 62K □ 50K，约 **19% 损耗**。换来”755K 请求 □ 1 笔订单” 的强幂等。
- 损耗源：每请求一次 Redis Lua + responseRecorder 拷贝。
- CircuitBreaker Closed 态**零锁开销**（仅 atomic.Load），吞吐影响可忽略。

stressTest/REPORT.md 链路压测表前 7 行

Idempotency 的极限场景：755K 请求 → 1 笔订单

- 50 VU × 15s 用**同一个 IK** 反复打 `/orders/create`。
- 累计 **755,033 次请求**，DB 实际订单数 **= 1**：
  - `SELECT COUNT(*) FROM `order` WHERE user_id=10 AND created_at > NOW() - 2 MIN` → 1
- 第一次走完业务后，后续全部由 Lua 读 `done` 态直接回放 → 50K RPS。
- **业务承诺**：客户端在熔断反复触发期间无脑重试 10K 次，仍只扣一次款。

> 重试是客户端的权利，幂等是服务端的义务。两者共同维护” 恰好一次”。

stressTest/REPORT.md §4 幂等中间件压测

把 paydown 的设计翻译成业务承诺

技术选择	业务承诺
AES-128-CBC + 双密钥	DBA 拿 dump 也看不到余额
6 位支付密码不存盘	整库泄漏没密码也解不开
单次 TX 五件事原子	不会出现” 扣了钱没改订单”
outbox 在 TX 内写	不会出现” 扣了钱下游没收到”
CircuitBreaker 5/10s/3	第三方挂了 5% 用户受影响，不是 100%
Idempotency 全程伴随	重试 10K 次也只扣一次

六条产品 / 运营

/ 客服三方都能背的话术，每条都在代码里找得到位置。*routes/router.go:114–121 + service/payment.go:35–200*

面试 Q&A

Q1：金额为什么不直接 `decimal` 存?

A：合规要求” 敏感字段不得明文落盘”。AES + 服务端密钥 + 用户密码双因子，比 decimal 多一道防线。

Q2：熔断打开期间客户端怎么处理?

A：拿 70002 □ 指数退避 (1s/2s/4s) □ 重试**必须带原 Idempotency-Key**。第一次若其实成功，第二次回放原结果不会二次扣款。

Q3：为何不用 Redis 分布式锁做幂等，要用 Lua?

A：分布式锁只解决” 不重入”，不解决” 结果回放”。Lua 是真状态机 (init/processing/done)：第一次走完后，同 IK 请求直接 Lua 内回放，**不进业务代码**。

Q4：密码错和余额不足为什么都返回” 金额不足”?

A：密码错 □ 乱码 □ int64 几乎不够 □ 走金额不足分支。**有意为之**：防穷举区分。

代码位置一览

PayDown	*service/payment.go:35–200*
AES 加解密	*repository/db/model/user.go:59–79*；*pkg/utils/encryption/money_encrypt.go*
CircuitBreaker	*middleware/circuitbreaker.go:44–128*
Idempotency	*middleware/idempotency.go:17–112*；*repository/cache/idempotency.go:32–90*
paydown 路由	*routes/router.go:114–121*
支付失败 Saga	*service/order_cancel.go:14–62*
业务码 + 压测	*pkg/e/code.go:53–58*；*stressTest/REPORT.md §4–5*

配套：deck 01 (幂等) / deck 04 (outbox+saga) / deck 15 (关单衔接)。